FUNSCIENTIA · KINGDOMS OF DISCOVERY

Online Saves with Save Codes

Backend task guide: what to build, how to build it step by step, and how to deliver it for review

K7QM-4FXA

A save code: type it on any device to continue the adventure.

For	**Djihane**, backend
From	Hicham (game development), with Dounia (project lead)
Date	22 October 2026 · version 1
Please deliver by	**Sunday 25 October 2026** (the Thirduni deadline is 31 October)
Time needed	about 2 to 3 days of work
Comes with	`Funscientia_Online_Saves_Kit.zip`: every file in this guide, ready to use

Contents

1. The task in one page
2. How it works
3. The progress data
4. Before you start
5. Step 1: Create your Supabase project
6. Step 2: Build the database (the SQL, explained)
7. Step 3: Test everything
8. Step 4 (optional): Statistics
9. Step 5: Deliver your work
10. Troubleshooting
11. How we review, and what comes next
12. Appendix: the full files

1. The task in one page

Funscientia is our web game about the human body. Today the game saves the player's progress only *inside their browser*: if a Thirduni jury member starts on a phone and continues on a laptop, they must start again. There is no server at all; the game is a static website on GitHub Pages.

Your task: build the small backend that saves progress **online**, found by a **save code**, like the passwords of old console games. No accounts, no emails, no passwords.

What the player sees

- When they start, the game shows their code, e.g. `K7QM-4FXA` (always visible in their profile).
- On another device, they click **"Continue with my code"**, type it, and carry on.
- We can also give the jury ready-made codes that start at chapter 2 or 3.

What you build (in Supabase)

- One table, `saves`.
- Three database functions: `create_save`, `save_progress`, `load_progress`.
- Security: the table is locked; only those three functions are open.
- A test page that proves it all works (14 automatic checks).

Good news: you write no server code

We use **Supabase**: a hosted PostgreSQL database (free plan) that automatically gives every database function a web address. The game calls those functions directly from the browser. So your work is **SQL** (three files, all given and explained here), plus **testing**, plus a clean **delivery**. This guide gives you everything; your job is to build it in your own project, understand it, prove it works, and hand it over.

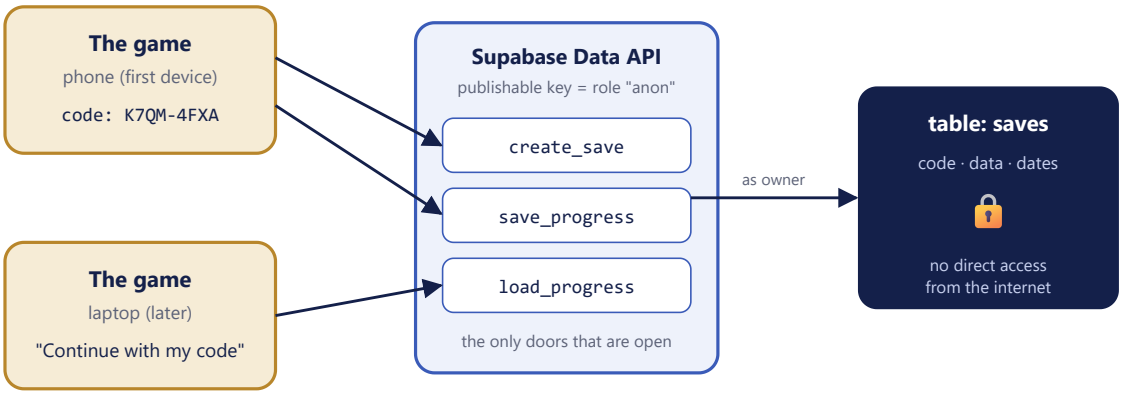
The plan

Step	What you do	Time
1. Supabase project	Create a free account and a test project	30 min
2. Build the database	Run three SQL files, and read the explanations	2 to 3 hours
3. Test	Open the test page, run the 14 checks, take screenshots	1 hour
4. Statistics (optional)	Run the totals queries	30 min
5. Deliver	Upload the `backend` folder to GitHub as a pull request	1 hour

> **Nothing you do can break the game.** You work in *your own* Supabase project and on *your own* branch. The game only uses your work after we review it together.

> **Three golden rules**
>
> 1. Never put a **password** or the **secret key** in any file, screenshot or message. (The repository is public.)
> 2. Never store personal data: no emails, phone numbers or addresses. The only name in a save is the nickname the child typed in the game.
> 3. Do not deliver if any of the 14 checks fails. Ask instead: we are happy to help.

2. How it works



The game never touches the table. It can only call three functions, and the functions check everything before touching the table.

The three moments

Moment	The game calls	What happens
A player starts	`create_save(data)`	A new row is created with a new random code. The function returns the code; the game shows it and remembers it.
The player makes progress	`save_progress(code, data)`	The row with that code gets the new data. Returns `true`, or `false` if there is no such code.
"Continue with my code"	`load_progress(code)`	Returns the saved data, or `null` if the code does not exist. The game then continues from there.

This is how the game will call them (JavaScript)

You do not need to write this; it shows how your functions are used. `rpc` means "call a database function".

```
// connect with the project URL and the PUBLISHABLE key (safe to put in a public website)
const db = supabase.createClient(PROJECT_URL, PUBLISHABLE_KEY);

// a new player: get a code
const { data: code, error } = await db.rpc('create_save', { p_data: progress });

// whenever progress changes (a few times per chapter)
await db.rpc('save_progress', { p_code: code, p_data: progress });

// on another device
const { data: saved } = await db.rpc('load_progress', { p_code: 'k7qm 4fxa' }); // case and spaces
don't matter
```

Why it is safe

- **The table is locked.** Row Level Security is on with no rules, and the API roles have no rights on it. Nobody can list, read, change or delete rows from the internet.
- **You must know a code to load a save.** Codes are 8 random characters from 31 possibilities: about **850 billion** codes, so guessing one is hopeless.

- **Inputs are checked.** The data must be a JSON object with a `version` and at most 16 000 bytes; codes are cleaned and must have the right shape. Nothing typed by a player is ever pasted into SQL.
- **Easy codes.** The alphabet has no `0 O 1 I L`, so children do not confuse characters.

3. The progress data

For the database, the progress is one **JSON object** (stored as `jsonb`). Your functions do not need to understand it; they only check that it is an object, that it has a `version`, and that it is not too big. Here is what the game will send (version 1):

```
{
  "version": 1,
  "hero": "nesma",
  "playerName": "Sara",
  "lang": "ar",
  "place": "esophagus",
  "discoveries": ["saliva_smell", "teeth_chew", "tongue_mix"],
  "predictions": { "saliva": "sugar", "pepsin": "protein" },
  "savedAt": "2026-10-23T10:15:00Z"
}
```

Field	Type	Meaning
`version`	number (always 1 for now)	The format of this object. If we change the format later, the number goes up.
`hero`	`"walid"`, `"nesma"`, `"fox"` or null	The avatar the player chose.
`playerName`	text, up to 20 characters	The nickname the player typed ("" = the character's own name). The only personal-ish field.
`lang`	`"en"` or `"ar"`	English or Arabic.
`place`	text	Where to continue: `overlook`, `road`, `road2`, `mouth`, `esophagus`, `stomach` …
`discoveries`	list of texts	Science facts the player unlocked, e.g. `saliva_amylase`.
`predictions`	object: question → answer	What the player predicted before each mini-game.
`savedAt`	date-time text (ISO 8601)	When the game saved it (the database also keeps its own dates).

Why one JSON object and not many columns? The game will grow (new chapters, new mini-games). With one `jsonb` column, the database never needs to change when the game adds a field. Postgres can still look inside it, as the statistics in step 4 show.

Two devices at once? If the same code is used on two devices at the same time, the last save wins. That is fine for us; you do not need to handle it.

4. Before you start

What you need

- ☐ A computer with a recent browser (Chrome, Edge or Firefox).
- ☐ A **GitHub** account (to sign in to Supabase and to deliver). Send Hicham your GitHub username, so you can be added to the repository. You will get an invitation e-mail from GitHub: accept it.
- ☐ The kit: `Funscientia_Online_Saves_Kit.zip`. Unzip it; you get a folder `backend` with all the files.
- ☐ Optional: a code editor such as **VS Code**, to read the files comfortably.

Words you will meet

Word	Meaning here
Supabase	A service that hosts a PostgreSQL database for us and puts a web API in front of it, with a dashboard in the browser.
SQL Editor	The page in the Supabase dashboard where you paste SQL and run it.
`jsonb`	A Postgres column type that stores a JSON object efficiently.
Function / RPC	A small program stored in the database. "RPC" (remote procedure call) = calling it from the web by its name.
Role `anon`	Who the game "is" for the database when it connects with the publishable key: an anonymous visitor.
Publishable key	The public key the game uses. Safe in a website, because the database decides what `anon` may do. Starts with `sb_publishable_` (older projects call it the "anon public" key).
Secret key	Starts with `sb_secret_` (older: "service_role"). It skips all security. **You do not need it. Never copy it anywhere.**
Row Level Security (RLS)	Postgres rules that decide which rows a role may see or change. RLS on with no rules = nothing is allowed.
`security definer`	A function that runs with the rights of its owner (not of the caller). That is how a locked table can still be used, but only in the ways the function allows.
`search_path`	Where Postgres looks for names like `saves`. We set it to empty in our functions and write full names (`public.saves`), so nobody can trick a function with a look-alike table.

5. Step 1: Create your Supabase project

Supabase sometimes renames buttons. If something looks a little different, look for the closest match; the ideas stay the same.

1. Go to **supabase.com** and click **Start your project**. Sign in with **GitHub**.
2. If asked, create an **organization** (your name is fine) on the **Free** plan.
3. Click **New project** and fill in:
   - **Project name:** `funscientia-saves-test`
   - **Database password:** click *Generate a password* and keep it in a safe place (a password manager). You will not need it for this task. **Never share it or put it in a file.**
   - **Region:** the closest to us, for example *Central EU (Frankfurt)* or *West EU*.
   - **Security options:** keep **Enable Data API** turned on (the game needs it). You may see **Automatically expose new tables and functions**: leave it **unticked**, which is the safer default since mid-2026. Our `security.sql` grants exactly what the game needs, so it works either way.
4. Click **Create new project** and wait 1 to 2 minutes while it starts.
5. Find and write down two things (you will paste them into the test page):
   - The **Project URL**, like `https://abcdefghijkl.supabase.co`, and the **Publishable key**, which starts with `sb_publishable_`. The easiest place is the **Connect** button at the top of the dashboard: it shows both. The keys are also in **Project Settings → API Keys**. If no publishable key is shown there, click **Create new API keys**. (Older projects call it the **anon public** key; Supabase is retiring those by the end of 2026.)

> **Careful:** the same page also shows a **secret** key (`sb_secret_…` or `service_role`). Do not copy that one. The test page refuses it on purpose.

> **Good to know:** free projects are **paused after about a week without activity**. Supabase e-mails a warning first. If your project is paused, open it in the dashboard and click **Resume project**; nothing is lost.

6. Step 2: Build the database

You will run three files from the kit, **in this order**: `schema.sql`, then `functions.sql`, then `security.sql`. For each file:

1. In the dashboard's left menu, open **SQL Editor** and start a **New query** (the **+** button).
2. Open the file from the kit in a text editor, select all (Ctrl + A), copy, and paste it into the query.
3. Click **Run** (or Ctrl + Enter). You should see **"Success. No rows returned"**. If the editor warns that the new table has no Row Level Security (when you run `schema.sql`), either choice is fine: `security.sql` turns RLS on anyway. For any other confirmation, read the message, then confirm.

> **Please copy from the kit files, not from this PDF.** PDFs can change quotes, dashes and line breaks. The listings here and in the appendix are for reading and understanding.

6.1 `schema.sql`: the table

```
create table public.saves (
  code        text primary key
              check (code ~ '^[2-9A-HJKMNP-Z]{4}-[2-9A-HJKMNP-Z]{4}$'),
  data        jsonb not null,
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now(),
  save_count  integer not null default 1
);
```

- `code` is the **primary key**: unique, and indexed, so finding a save by its code is instant.
- The `check` uses a **regular expression** (`~`): 4 allowed characters, a dash, 4 allowed characters. `[2-9A-HJKMNP-Z]` means "2 to 9, A to H, J, K, M, N, P to Z", which is our 31-character alphabet without 0, O, 1, I and L. Even a mistake in a function could never store a badly shaped code.
- `data` holds the progress object (`jsonb`), and can never be empty (`not null`).
- `created_at` / `updated_at` are filled automatically with the current time (`default now()`). `timestamptz` stores the time with its time zone.
- `save_count` counts the saves: handy to see that a code is really being used.

6.2 `functions.sql`: the three doors

All three functions begin the same way:

```
create or replace function public.create_save(p_data jsonb)
returns text
language plpgsql
security definer
set search_path = ''
as $$ ... $$;
```

- `create or replace`: running the file again simply updates the function.
- `p_data jsonb`: the parameter. The game passes it by name: `{ p_data: … }`. Parameter names start with `p_`, local variables with `v_`, so they never clash with column names.
- `language plpgsql`: Postgres's own programming language (with `if`, `loop`, variables).

- `security definer` and `set search_path = ''`: explained in section 4. This pair is Supabase's recommended safe pattern.
- `$$ … $$`: the body of the function, written between two `$$` marks.

`create_save(p_data)` → the new code

```
-- 1. check the data: a JSON object, with a "version", not too big
if p_data is null or jsonb_typeof(p_data) <> 'object' or not (p_data ? 'version') then
  raise exception 'data must be a JSON object with a "version"' using errcode = '22023';
end if;
if octet_length(p_data::text) > 16000 then
  raise exception 'data is too big (max 16000 bytes)' using errcode = '22023';
end if;
```

- `jsonb_typeof` tells if the JSON is an object, an array, a number… `p_data ? 'version'` means "the object has a key named version".
- `octet_length(p_data::text)` is the size in bytes. Real saves are under 2 000 bytes, so 16 000 leaves plenty of room while stopping abuse.
- `raise exception` stops the function and returns an error to the caller (nothing is written). `22023` is Postgres's code for "invalid parameter value".

```
-- 2. make a random code; if it is already taken (very rare), try again
for attempt in 1..10 loop
  v_bytes := uuid_send(gen_random_uuid());   -- 16 random bytes
  v_code := '';
  for i in 1..8 loop
    if i = 5 then v_code := v_code || '-'; end if;
    v_code := v_code || substr(v_alphabet, get_byte(v_bytes, v_picks[i]) % 31 + 1, 1);
  end loop;
  begin
    insert into public.saves (code, data) values (v_code, p_data);
    return v_code;
  exception when unique_violation then
    -- that code is taken: loop and try another one
  end;
end loop;
```

- `gen_random_uuid()` is built into Postgres and is **cryptographically random** (unlike `random()`, which is predictable). `uuid_send` turns it into 16 bytes.
- For each of the 8 characters we take one byte (a number 0 to 255), keep the remainder of dividing by 31 (`% 31`, giving 0 to 30), and pick that character of the alphabet (`substr` counts from 1, hence `+ 1`). `||` joins texts.
- `v_picks` lists which bytes we use. A UUID has a few fixed bits (bytes 6 and 8 hold its "version" and "variant" marks), so we skip those bytes.
- The inner `begin … exception … end` catches the "duplicate primary key" error and simply tries a new code. After 10 failures (practically impossible) it gives up with an error.

`save_progress(p_code, p_data)` → true / false

```
-- accept 'k7qm4fxa', 'K7QM 4FXA', 'k7qm-4fxa'...
v_code := upper(regexp_replace(coalesce(p_code, ''), '[^A-Za-z0-9]', '', 'g'));
if v_code !~ '^[2-9A-HJKMNP-Z]{8}$' then
  return false;
end if;
v_code := substr(v_code, 1, 4) || '-' || substr(v_code, 5, 4);

update public.saves
   set data = p_data, updated_at = now(), save_count = save_count + 1
 where code = v_code;
return found;
```

- It first checks the data exactly like `create_save`.
- **Cleaning the code:** `coalesce` turns a missing code into `''`; `regexp_replace(…, '[^A-Za-z0-9]', '', 'g')` removes everything that is not a letter or digit (spaces, dashes); `upper` makes it upper-case. If what is left is not 8 allowed characters, the answer is simply `false`. Then the dash is put back.
- `found` is a built-in variable: `true` if the `update` changed a row.

`load_progress(p_code)` → the data, or null

```
select s.data into v_data from public.saves s where s.code = v_code;
return v_data;  -- null when not found
```

- It cleans the code the same way (a bad code returns `null`, not an error), then reads the row. `stable` tells Postgres the function only reads.
- The player's code is always passed as a **parameter**, never glued into the SQL text, so "SQL injection" is impossible. The test page even tries `x'; drop table saves; --`.

6.3 `security.sql`: lock the table, open three doors

```
alter table public.saves enable row level security;
revoke all on table public.saves from anon, authenticated;

revoke execute on function public.create_save(jsonb)         from public, anon, authenticated;
revoke execute on function public.save_progress(text, jsonb) from public, anon, authenticated;
revoke execute on function public.load_progress(text)        from public, anon, authenticated;

grant execute on function public.create_save(jsonb)         to anon;
grant execute on function public.save_progress(text, jsonb) to anon;
grant execute on function public.load_progress(text)        to anon;
```

- **RLS on, no policies:** through the API, the table looks empty and cannot be changed. Supabase recommends RLS on every table in the `public` schema.
- `revoke all` **on the table:** a second lock. Even if someone later added a policy by mistake, `anon` still has no rights.
- **Functions:** in Postgres, everyone may run a new function by default (`public` = every role). We first close them for everybody, then open exactly our three to `anon`, the role of the publishable key (the game). The game has no logins, so `authenticated` (logged-in users) gets nothing. In a `grant`, a function is named with its parameter types, e.g. `save_progress(text, jsonb)`.

- **Why revoke from `public` too?** Revoking only from `anon` is not enough: `anon` would still run the function through `public`, which every role belongs to. Supabase's own advice is exactly this pattern: revoke from `public`, `anon` and `authenticated`, then grant back only the roles that should call it.
- The functions still work on the locked table because they are `security definer`: they run as their owner (`postgres`), and the table's owner is not stopped by RLS.

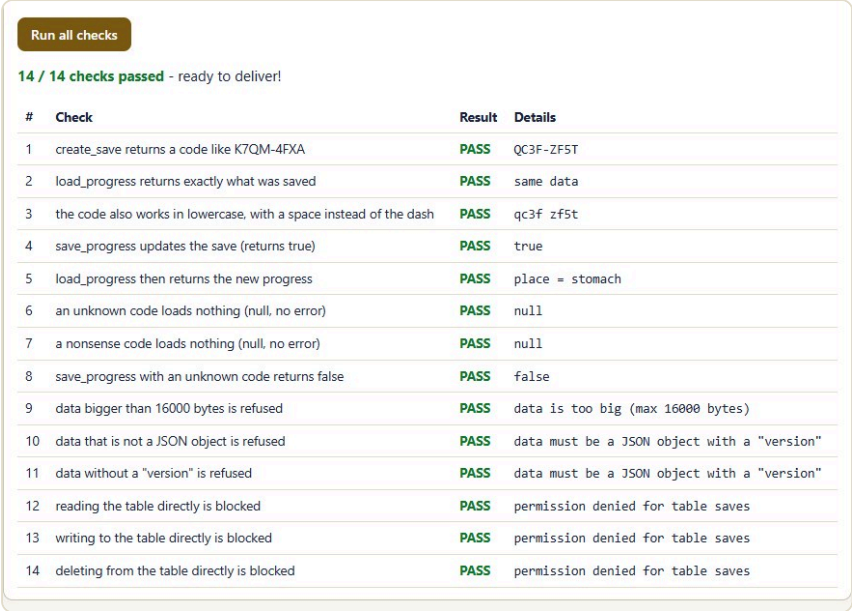
6.4 Look at your work in the dashboard

- ☐ **Table Editor → saves**: the table exists and is marked **RLS enabled**. It is empty for now.
- ☐ **Database → Functions**: you see `create_save`, `save_progress`, `load_progress`.
- ☐ **Advisors → Security Advisor**: run it, compare with the table below, and take a screenshot.

What the Security Advisor shows	Level	Is it OK?
Public Can Execute SECURITY DEFINER Function (lint 0028), once for each of our three functions	WARN	**Yes, expected and intended.** The warning means "anonymous visitors can run this owner-rights function". That is the whole point of our three doors, and each one only touches the save whose code you give it. Write one line about it in your README.
RLS Enabled No Policy on `saves` (lint 0008)	INFO	**Yes, intended:** that is exactly how the table is locked.
Function Search Path Mutable (0011) on our functions	WARN	**No.** `functions.sql` was changed or not run as given. Run it again.
RLS Disabled in Public (0013) on `saves`	ERROR	**No.** `security.sql` was not run. Run it, then the checks.
… *SECURITY DEFINER Function* for **signed-in** users (0029)	WARN	**No** with this guide's `security.sql` (it gives the functions to `anon` only). Run it again.
Anything about Auth, e-mails or passwords	any	Not part of this task; ignore it.

7. Step 3: Test everything

1. Open `test.html` from the kit in your browser (double-click it). If your browser blocks it, open the `backend` folder in VS Code and use the **Live Server** extension.
2. Paste your **Project URL** and **Publishable key** and click **Connect**. They are saved only in your browser, never in the file.
3. **Try it by hand:** click **create_save**. You get a code (it appears in the "Save code" box). Change something in the JSON, e.g. `"place": "stomach"`, and click **save_progress** (answer: `true`). Click **load_progress**: you get your new data back.
4. In the dashboard, open **Table Editor → saves**: your saves are there, with their codes and dates. (As the owner, the dashboard can see the table; the internet cannot.)
5. Click **Run all checks**. All **14 checks must PASS**.
6. Take a screenshot of the checks table (crop away the key fields at the top).



What success looks like: 14 / 14 checks passed.

The 14 checks

#	Check	Proves that…
1	create_save returns a code like K7QM-4FXA	codes are made and well shaped
2	load_progress returns exactly what was saved	nothing is lost or changed
3	the code works in lowercase, with a space	children can type it any way
4–5	save_progress updates, load then returns it	progress really updates
6–7	unknown or nonsense code → null, no error	bad codes are harmless (even an injection attempt)
8	save_progress with an unknown code → false	nobody can create codes of their own choosing
9–11	too big, not an object, no version → refused	the input checks work
12–14	direct read / write / delete on the table → blocked	**the lock works**: the most important checks

> **If check 12, 13 or 14 fails**, the table is open to the internet. Run `security.sql` again and re-test. Never deliver with a failing check.

8. Step 4 (optional): Statistics

Because all progress is in one table, we get learning statistics almost for free. `stats.sql` contains four queries; run them in the **SQL Editor** (you can run them one at a time by selecting a query, then Run):

- how many players, and how many came back to play again;
- which discoveries were unlocked, and by how many players;
- what players predicted before each mini-game (useful for our report to the jury);
- how far players got, and in which language.

They use `jsonb_array_elements_text` (one row per item of a JSON list) and `jsonb_each_text` (one row per key/value of a JSON object), and `->` / `->>` to read a field as JSON / as text. Create a few saves with the test

page first so there is something to count.

> **These queries are for the team only.** They are not functions, so they cannot be reached from the internet. Please do not turn them into API functions.

9. Step 5: Deliver your work

What to deliver: one folder, `backend`

```
backend/
  README.md            your notes (a template is in the kit; write it in your own words)
  schema.sql
  functions.sql
  security.sql
  stats.sql            (optional)
  test.html
  screenshots/
    checks.png         the 14 checks, all PASS
    table-editor.png   the saves table with a few test rows
    security-advisor.png
```

Final checklist

- ☐ All 14 checks PASS on your project.
- ☐ `README.md` filled in: results, a short "security in your words", and any change you made (and why).
- ☐ No password, no secret key, no publishable key in any file or screenshot. (Search the folder for `sb_` to be sure.)
- ☐ If you changed any SQL, you ran all the checks again after the change.

Option A (easiest): upload on the GitHub website

1. Open **github.com/NinjaTraderCodes/Funscientia**. (Accept the collaborator invitation from GitHub first. If you have none, click **Fork** and do the same steps in your fork.)
2. Click the branch menu (it shows `main`), type `backend-saves` and click **Create branch backend-saves from main**.
3. With `backend-saves` selected, click **Add file** → **Upload files** and drag your whole `backend` folder onto the page (GitHub keeps the folder structure).
4. Write a message such as *Online saves: table, functions, security, tests*, choose **Commit directly to the backend-saves branch**, and click **Commit changes**.
5. Click **Compare & pull request** (or **Pull requests** → **New pull request**, base `main` ← compare `backend-saves`). Paste the description below and click **Create pull request**.

Option B: with Git on your computer

```
git clone https://github.com/NinjaTraderCodes/Funscientia.git
cd Funscientia
git checkout -b backend-saves
# copy your backend folder here, then:
git add backend
git commit -m "Online saves: table, functions, security, tests"
git push -u origin backend-saves
# then open the pull request on GitHub
```

The repository holds the whole game (images, videos), so cloning takes a while. Option A avoids that.

Option C: if GitHub gives you trouble

Zip the `backend` folder and send it to Hicham in the team chat. That is perfectly fine.

Pull request description (copy and fill in)

```
Online saves with save codes (backend)

What: table saves; functions create_save / save_progress / load_progress;
security (table locked, only the 3 functions open); test page; stats queries.

Tested on my Supabase project: 14 / 14 checks passed (screenshots/checks.png).
Security Advisor: only the expected 0028 warnings and the 0008 info (screenshots/security-
advisor.png).

Changes from the guide: none  /  <what and why>
Questions: <anything>
```

10. Troubleshooting

You see…	What it means / what to do
`relation "saves" already exists`	You ran `schema.sql` twice; the table is already there. Carry on with the next file. To start completely fresh **in your test project only**: run `drop table public.saves;` (it deletes all saves), then run the three files again.
`permission denied for function create_save`	`security.sql` was not run, or ran before `functions.sql`. Run `security.sql` again.
`Could not find the function public.create_save(p_data) in the schema cache`	The API has not noticed the new functions yet. Wait a minute, or run `notify pgrst, 'reload schema';` in the SQL Editor. Also check that the names in the call match exactly.
`Invalid API key`, or status 401	The URL or key is wrong, or has a space. Copy the **publishable** key again from **Project Settings → API Keys**.
`Failed to fetch`	No internet, a typo in the URL, or the project is **paused** (open it in the dashboard and click **Resume project**).
Check 12, 13 or 14 FAILS (rows visible, or insert accepted)	The table is not locked. Run `security.sql` again, then re-run all checks. Do not deliver until they pass.
Check 2 or 3 fails	Did you change the functions? Compare with the kit files and run `functions.sql` again.
The test page says "STOP: that is a SECRET key"	You pasted the secret key. Use the publishable one, and never paste the secret key anywhere.
Red `400` / `401` lines in the browser console while the checks run	Expected: checks 9 to 14 try forbidden things on purpose, and the server says no. Only the PASS / FAIL column matters.
Anything else	Send Hicham a screenshot of the error (crop out any key). Being stuck for more than 30 minutes is a good reason to ask.

11. How we review, and what comes next

Our review (Monday 26 October)

1. We read your files and your README; any change from the guide is discussed with you.
2. We run your three files in a **fresh** Supabase project and open your `test.html` against it: 14 / 14 must pass for us too.
3. We run the Security Advisor and compare it with the table in section 6.4.
4. We either approve the pull request, or comment on it with what to adjust (then you push the fixes to the same branch).

After approval (small part B, Tuesday 27 October)

- The team creates the **real** project, `funscientia`, and invites you to it. You run the three files there and the 14 checks once more.

- We send you the JSON of three "jury" starting points (chapter 1, 2, 3). You create them with the test page and send us the three codes.
- Hicham connects the game (Wednesday 28 to Thursday 29 October): the code in the profile, "Continue with my code" on the start page, and automatic saving. The browser save stays as a backup, so the game still works if the server is unreachable.

Ideas for later (not now)

Deleting saves unused for a year · class codes for teachers · a small dashboard page for the statistics. If one interests you, write it in the "Questions / ideas" part of your README.

> **Thank you, Djihane!** This is the piece that lets the jury play on any device and continue where they stopped. It is also the first backend of Funscientia. Questions are always welcome in the team chat.

Appendix: the full files

For reading. To run them, use the files in the kit (copying from a PDF can change characters). `test.html` and `README.md` are in the kit only.

schema.sql

```
-- schema.sql: the table that holds every online save.
-- One row = one player's progress, found by its save code.

create table public.saves (
  -- the save code the player sees, e.g. 'K7QM-4FXA'
  -- (8 characters, no 0/O, 1/I or L so it is easy to read and type)
  code        text primary key
              check (code ~ '^[2-9A-HJKMNP-Z]{4}-[2-9A-HJKMNP-Z]{4}$'),
  -- the progress itself, as the game sends it (a JSON object)
  data        jsonb not null,
  created_at  timestamptz not null default now(),
  updated_at  timestamptz not null default now(),
  -- how many times the game saved (handy when reviewing)
  save_count  integer not null default 1
);

comment on table public.saves is
  'Funscientia online saves. Only reachable through create_save, save_progress and load_progress.';
```

functions.sql

```
-- functions.sql: the only three ways the game talks to the database.

-- 1) create_save(data) -> a new save code, e.g. 'K7QM-4FXA'
create or replace function public.create_save(p_data jsonb)
returns text
language plpgsql
security definer          -- runs with the owner's rights, so the table can stay locked
set search_path = ''      -- safety: every name below is written in full (public.saves)
as $$
declare
  v_alphabet constant text := '23456789ABCDEFGHJKMNPQRSTUVWXYZ'; -- 31 easy-to-read characters
  v_picks    constant int[] := array[0, 1, 2, 3, 4, 5, 10, 11];  -- random bytes of a UUID
  v_bytes    bytea;
  v_code     text;
begin
  -- the data must be a JSON object, with a version number, and not too big
  if p_data is null or jsonb_typeof(p_data) <> 'object' or not (p_data ? 'version') then
    raise exception 'data must be a JSON object with a "version"' using errcode = '22023';
  end if;
  if octet_length(p_data::text) > 16000 then
    raise exception 'data is too big (max 16000 bytes)' using errcode = '22023';
  end if;

  -- pick a random code; in the very rare case it already exists, try again
  for attempt in 1..10 loop
    v_bytes := uuid_send(gen_random_uuid());  -- 16 cryptographically random bytes
    v_code := '';
    for i in 1..8 loop
```

```
      if i = 5 then
        v_code := v_code || '-';
      end if;
      v_code := v_code || substr(v_alphabet, get_byte(v_bytes, v_picks[i]) % 31 + 1, 1);
    end loop;
    begin
      insert into public.saves (code, data) values (v_code, p_data);
      return v_code;
    exception when unique_violation then
      -- that code is taken: loop and try another one
    end;
  end loop;
  raise exception 'could not create a save code, please try again';
end;
$$;


-- 2) save_progress(code, data) -> true if saved, false if there is no such code
create or replace function public.save_progress(p_code text, p_data jsonb)
returns boolean
language plpgsql
security definer
set search_path = ''
as $$
declare
  v_code text;
begin
  if p_data is null or jsonb_typeof(p_data) <> 'object' or not (p_data ? 'version') then
    raise exception 'data must be a JSON object with a "version"' using errcode = '22023';
  end if;
  if octet_length(p_data::text) > 16000 then
    raise exception 'data is too big (max 16000 bytes)' using errcode = '22023';
  end if;

  -- accept 'k7qm4fxa', 'K7QM 4FXA', 'k7qm-4fxa'...: keep letters/digits, upper-case, add the dash
  v_code := upper(regexp_replace(coalesce(p_code, ''), '[^A-Za-z0-9]', '', 'g'));
  if v_code !~ '^[2-9A-HJKMNP-Z]{8}$' then
    return false;
  end if;
  v_code := substr(v_code, 1, 4) || '-' || substr(v_code, 5, 4);

  update public.saves
     set data = p_data,
         updated_at = now(),
         save_count = save_count + 1
   where code = v_code;
  return found;  -- true when a row was updated
end;
$$;


-- 3) load_progress(code) -> the saved data, or null if there is no such code
create or replace function public.load_progress(p_code text)
returns jsonb
language plpgsql
stable                      -- only reads
security definer
set search_path = ''
as $$
```

```
declare
  v_code text;
  v_data jsonb;
begin
  v_code := upper(regexp_replace(coalesce(p_code, ''), '[^A-Za-z0-9]', '', 'g'));
  if v_code !~ '^[2-9A-HJKMNP-Z]{8}$' then
    return null;
  end if;
  v_code := substr(v_code, 1, 4) || '-' || substr(v_code, 5, 4);

  select s.data into v_data from public.saves s where s.code = v_code;
  return v_data;  -- null when not found
end;
$$;
```

security.sql

```
-- security.sql: lock the table; open only the three functions.

-- 1) Row Level Security ON, with no policies = nobody can read or write the table
--    through the API. (The functions still can: they run as the owner.)
alter table public.saves enable row level security;

-- 2) Belt and braces: take away every direct right on the table from the API roles.
revoke all on table public.saves from anon, authenticated;

-- 3) Postgres lets every role (PUBLIC) run a new function by default: close them first...
revoke execute on function public.create_save(jsonb)         from public, anon, authenticated;
revoke execute on function public.save_progress(text, jsonb) from public, anon, authenticated;
revoke execute on function public.load_progress(text)        from public, anon, authenticated;

-- ...then open exactly these three to the game: "anon" is the role of the publishable key.
-- (The game has no logins, so "authenticated" does not need them.)
grant execute on function public.create_save(jsonb)         to anon;
grant execute on function public.save_progress(text, jsonb) to anon;
grant execute on function public.load_progress(text)        to anon;
```

stats.sql

```
-- stats.sql (optional): totals for the team, run in the SQL Editor only.
-- Nothing here is reachable from the game or the internet.

-- How many players, and how many came back to play again?
select count(*)                                as players,
       count(*) filter (where save_count > 1)  as played_more_than_once,
       max(updated_at)                         as last_played
from public.saves;

-- Which discoveries were unlocked, and by how many players?
select d.discovery, count(*) as players
from public.saves s,
     jsonb_array_elements_text(s.data -> 'discoveries') as d(discovery)
group by d.discovery
order by players desc;

-- What did players predict before each mini-game?
select p.question, p.answer, count(*) as players
```

```
from public.saves s,
     jsonb_each_text(s.data -> 'predictions') as p(question, answer)
group by p.question, p.answer
order by p.question, players desc;

-- How far did players get, and in which language?
select s.data ->> 'place' as place, s.data ->> 'lang' as lang, count(*) as players
from public.saves s
group by 1, 2
order by players desc;
```